БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет прикладной математики и информатики

Кафедра технологий программирования

ОТЧЕТ

Лабораторная работа №4

«Стили интеграции программных систем и проектирование API»

по курсу «Проектирование человеко-машинных интерфейсов»

Выполнила: команда проекта *GremlinDrive*

Студенты группы 10а

Минск, 2026 г.

Содержание

1 Цель и исходные материалы **2**
- 1.1 Выбранный способ интеграции . . . 2
- 1.2 Состав комплекта . . . 2
- 1.3 Используемые инструменты . . . 3

2 Модель требований и варианты использования **3**
- 2.1 Уточнения требований для API . . . 5
- 2.2 Ключевые сценарии . . . 5

3 Логическая модель данных **5**
- 3.1 Ключевые правила . . . 6

4 Физическая модель данных **7**
- 4.1 Ограничения и индексы . . . 9
- 4.2 Фрагменты DDL . . . 9

5 Маршруты и контракт API **10**
- 5.1 Пример запроса и ответа . . . 11
- 5.2 Коды ответов . . . 12

6 Проверка API и схемы данных **12**
- 6.1 Воспроизведение . . . 12

7 Проектирование и реализация MCP **13**
- 7.1 Инструменты . . . 13
- 7.2 Ресурсы и промпты . . . 14
- 7.3 Проверка . . . 14

8 Вклад участников **14**

9 Выводы **15**

10 Список литературы **15**

1 Цель и исходные материалы

Цель работы — спроектировать серверный API и MCP-интерфейс GremlinDrive, продолжая требования и модели лабораторной работы № 3. Результат включает контракт на 52 HTTP-операции, логическую и физическую модели данных PostgreSQL, stateful mock на Python/SQLite и работающий MCP-сервер.

Работа выполнена по методическим указаниям «ПЧМИ ЛабРабота4 2026–2027». Для группы 10 обязательны задания 1–5. Локальный стенд и CI используются как средство проверки, но не подменяют промышленный backend.

Исходные материалы. SRS, User Stories и модели ЛР3 из коммита `392a849`, подготовленные Матвеем Вержбицким. Сохранены роли гостя, пользователя, владельца общей папки и администратора, а также сущности данных ЛР3. Идентификаторы US-01–US-16, FR-01–FR-30 и UC-01–UC-05 не переиспользованы под другие сценарии. UC-01 обозначает загрузку файла, UC-02 — работу со ссылкой, UC-03 — приглашение, UC-04 — восстановление версии, UC-05 — запрос квоты или прав. Дополнительные варианты начинаются с UC-06.

1.1 Выбранный способ интеграции

Для веб- и мобильного клиентов выбран REST/JSON через HTTPS. Клиенты не обращаются к базе данных напрямую: это связало бы их со схемой хранения и разнесло бы проверки прав по нескольким приложениям. Обмен файлами остаётся пользовательской функцией, а не способом синхронизации серверных таблиц. Длительные операции загрузки, проверки и уведомлений в целевой архитектуре выполняет worker с очередью Redis.

MCP-сервер добавляет интерфейс для LLM-клиента и повторно использует доменные проверки. Для учебного MVP выбраны официальный Python SDK и SQLite; целевой стек NestJS, PostgreSQL и MinIO из ЛР3 не считается реализованным этим прототипом.

1.2 Состав комплекта

Таблица 1 – Артефакты лабораторной работы №4

Артефакт	Назначение
`requirements.md`, `traceability.json`	Модель требований, 19 вариантов использования и трассировка на 52 операции
`api-design.md`, `openapi.json`	Маршруты, схемы запросов и ответов OpenAPI 3.1
`database/logical-model.md`	Логическая модель и связи сущностей
`database/schema.sql`	Физическая модель PostgreSQL, ограничения и индексы
`diagrams/`	Исходные тексты и изображения PlantUML
Backend-репозиторий	FastAPI mock, SQLite-хранилище, MCP-сервер и тесты
`bruno/`, `postman_collection.json`	Коллекции запросов для проверки контракта
`evidence/`	Протоколы pytest, Prism, PostgreSQL, Bruno и MCP

1.3 Используемые инструменты

Таблица 2 – Инструменты выполнения работы

Инструмент	Назначение	Адрес
PlantUML	Логическая, физическая модели, MCP и инфраструктура	https://plantuml.com
OpenAPI 3.1	Спецификация REST API	https://spec.openapis.org
FastAPI	Stateful mock на Python	https://fastapi.tiangolo.com
PostgreSQL 16	Целевая физическая СУБД	https://www.postgresql.org
Prism	Симуляция полного контракта	https://stoplight.io/prism
Bruno	Проверка запросов по коллекции	https://www.usebruno.com
MCP Python SDK	MCP-сервер по stdio	https://modelcontextprotocol.io
Docker Compose	Локальный стенд API, Prism и PostgreSQL	https://docs.docker.com/compose

2 Модель требований и варианты использования

Входной материал — SRS и User Stories ЛР3. Действующее лицо задаёт цель через пользовательскую историю, вариант использования описывает действия и результат, а конечная точка предоставляет серверный контракт. Сохранены UC-01–UC-05; для API добавлены UC-06–UC-19. Полная трассировка 52 операций хранится в `traceability.json`.

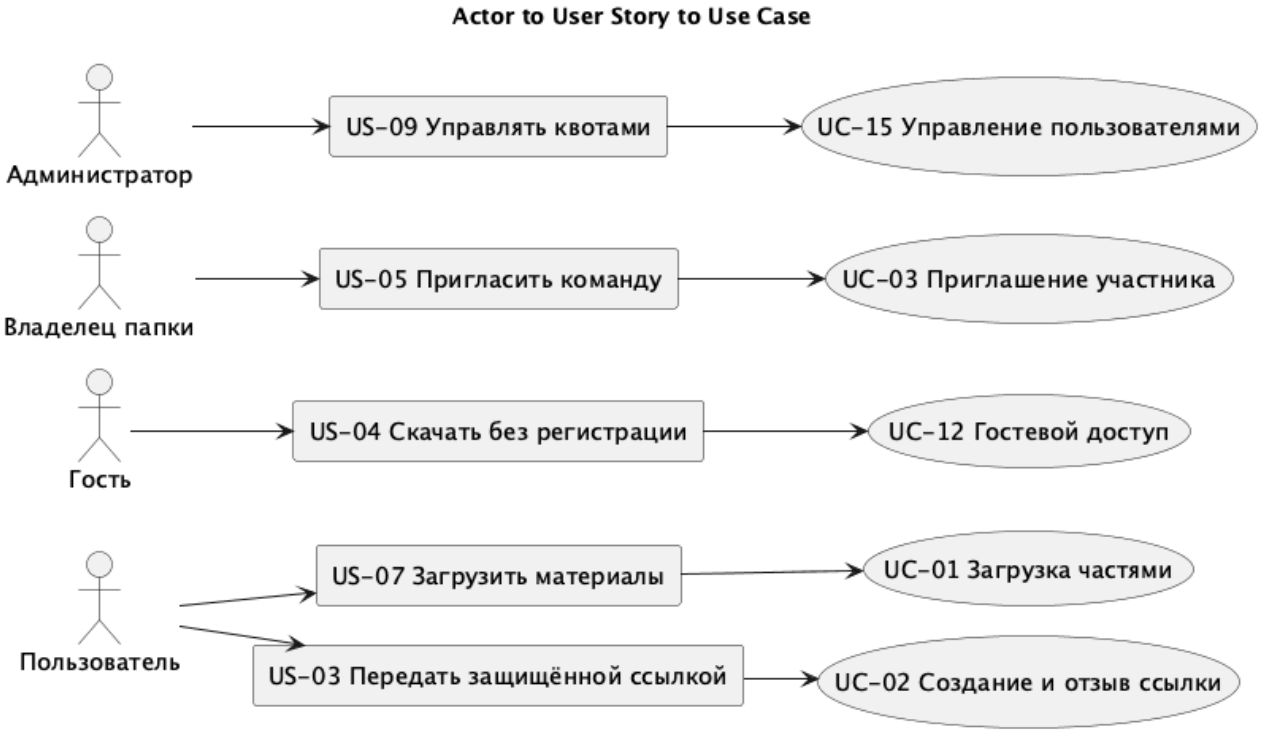


Рис. 1 – Базовые варианты использования UC-01–UC-05 и их связь с действующими лицами

На рисунке показаны базовые сценарии, продолжающие ЛР3. Расширенный реестр из 19 вариантов приведён в таблице 3; он покрывает регистрацию, сессии, TOTP, файлы, корзину, комментарии, уведомления, администрирование, LDAP и проверку окружения.

Таблица 3 – Реестр вариантов использования ЛР4

ID	**Действующее лицо**	**User Story**	**Вариант использования и результат**
UC-01	Пользователь	US-07	Загрузка файла частями; после завершения создана версия и начата проверка
UC-02	Пользователь	US-03, US-16	Создание, просмотр и отзыв ссылок; управляемый доступ к активному файлу
UC-03	Владелец папки	US-05	Приглашение и принятие приглашения; участник добавлен с ролью
UC-04	Участник папки	US-06	Просмотр и восстановление версий; создаётся следующая версия
UC-05	Пользователь и администратор	US-10, US-15	Запрос и решение по квоте; решение применяется однократно в транзакции
UC-06	Гость	US-01	Регистрация и подтверждение e-mail; аккаунт активирован
UC-07	Пользователь	US-02	Вход, выход и восстановление доступа; сессия выдана или отозвана
UC-08	Пользователь	US-12	Настройка и проверка TOTP; вход требует второй фактор
UC-09	Пользователь	US-07	Работа с файлами и папками; создана папка или изменено имя
UC-10	Пользователь	US-06, US-07	Поиск, метаданные и скачивание; возвращены доступные объекты
UC-11	Пользователь	US-11	Корзина и восстановление; старые ссылки не активируются
UC-12	Гость	US-04	Открытие ссылки и скачивание; проверены пароль, срок и уровень доступа
UC-13	Владелец папки	US-05	Изменение роли и удаление участника; новые права применяются при следующем запросе
UC-14	Участник папки	US-13	Комментарии и уведомления; комментарий сохранён, уведомление доставлено
UC-15	Администратор	US-09	Пользователи, роли и квоты; изменение проверено и записано в аудит
UC-16	Администратор	US-14	Аудит и экспорт отчётов; получен журнал или задача формирования отчёта
UC-17	Мобильный пользователь	US-07, US-08	Push, фоновая загрузка и синхронизация; клиент получил изменения после сети
UC-18	Администратор	US-09	Запуск синхронизации LDAP; сформирован результат синхронизации

ID	Действующее лицо	User Story	Вариант использования и результат
UC-19	Разработчик	US-09	Проверка окружения; получен статус работоспособности mock-сервиса

2.1 Уточнения требований для API

1. API-01. Пользователь устанавливается по сессии; идентификатор из тела запроса не меняет действующее лицо. Право администратора не даёт права читать содержимое личных файлов.
2. API-02. Ссылка не раскрывает метаданные до проверки пароля. После пяти неверных попыток доступ блокируется на 15 минут. Срок по умолчанию — 14 дней, `null` означает бессрочную ссылку.
3. API-03. Перенос в корзину отзывает ссылки. Восстановление файла не публикует его повторно. Карантин запрещает публикацию.
4. API-04. Завершение загрузки идемпотентно по ключу операции. Квота резервируется до передачи данных, а занятый объём меняется в транзакции.
5. API-05. Восстановление версии создаёт новую версию, не переписывая старую. Параллельная запись контролируется версией объекта или `If-Match`.
6. API-06. Клиент получает коды 401, 403, 404, 409, 422 и 429 с понятной причиной. Пароли, хеши и токены не попадают в обычные ответы и журнал аудита.
7. API-07. Тёмная тема, биометрия и офлайн-кэш относятся к клиенту; биометрические данные на сервер не отправляются.

2.2 Ключевые сценарии

UC-02. Передача файла по ссылке. Пользователь с правом редактирования отправляет `file_id`, уровень доступа, пароль и срок. Сервер проверяет права и состояние файла, создаёт случайный токен, сохраняет хеши токена и пароля и возвращает 201 с URL и датой окончания. При недостаточных правах возвращается 403, при корзине или карантине — 409, при коротком пароле — 422.

UC-12. Открытие гостевой ссылки. Гость отправляет пароль; после проверки сервер выдаёт краткоживущий токен доступа и метаданные. Скачивание требует уровень `download`. При уровне `view` скачивание отклоняется с 403, после пяти ошибок возвращается 429, истёкшая или отозванная ссылка возвращает 404 без имени файла.

UC-05. Решение по запросу квоты. Администратор открывает очередь, выбирает запрос и указывает новую квоту либо причину отказа. Сервер блокирует запись, проверяет состояние `pending`, меняет квоту и сохраняет решение одной транзакцией. Повторное решение даёт 409; уменьшение ниже занятого объёма запрещено.

3 Логическая модель данных

Логическая модель продолжает 16 сущностей ЛР3 и добавляет `upload_sessions` для возобновляемой загрузки и идемпотентного завершения. Итого — 17 сущностей. Содержимое файлов хранится в S3-совместимом хранилище, а в базе остаются метаданные, права, версии и ссылки.

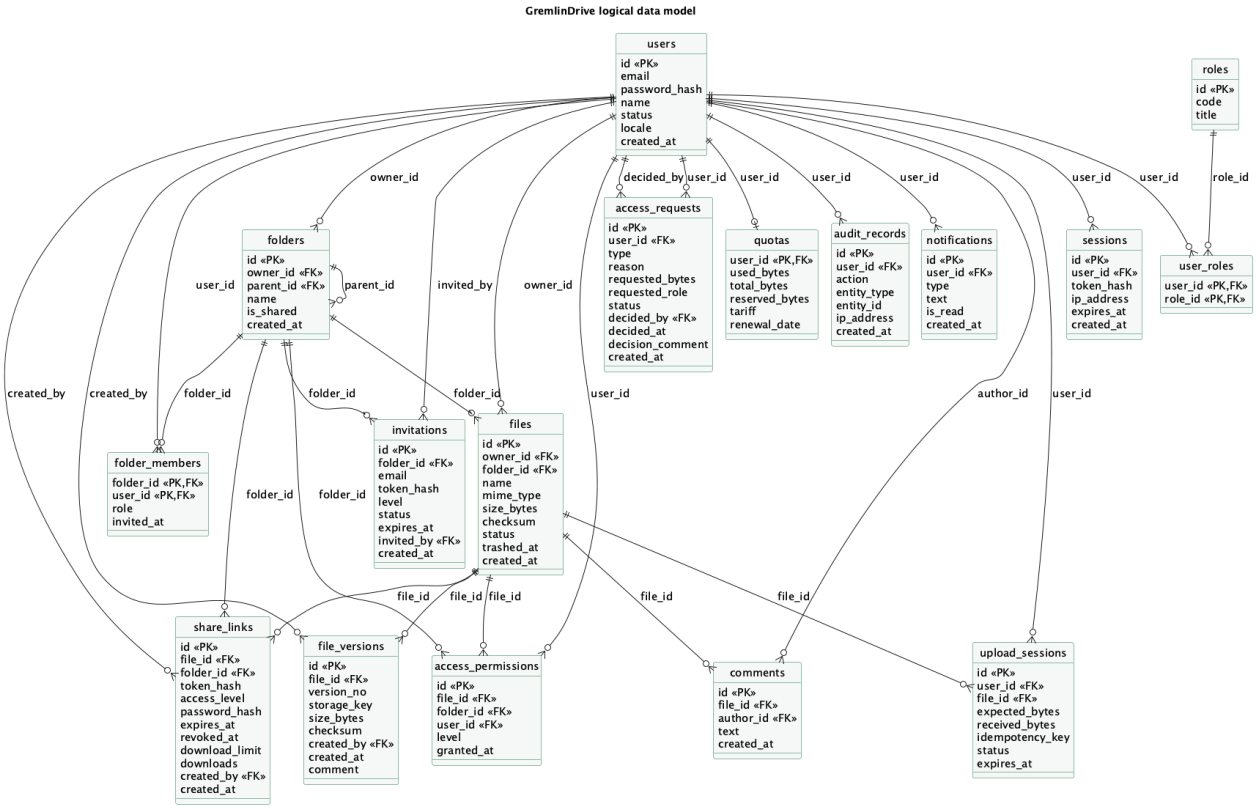


Рис. 2 – Логическая ER-модель GremlinDrive

Таблица 4 – Сущности логической модели

Сущность	Назначение и связи
`users`	Учётная запись, статус, локаль и время создания; владелец файлов и папок
`roles`, `user_roles`	Роли пользователя и связь многие-ко-многим
`sessions`	Серверные сессии; хранится хеш токена, IP и срок действия
`folders`	Иерархия папок; `parent_id` ссылается на родителя, владелец обязателен
`files`	Метаданные файла, папка, размер, checksum и состояние
`file_versions`	История версий; каждая версия ссылается на уникальный `storage_key`
`share_links`	Публичные ссылки на файл или папку; токен и пароль хранятся хешами
`access_permissions`	Индивидуальные права на файл или папку
`folder_members`	Участники общей папки и их роль
`invitations`	Приглашения по e-mail со сроком и статусом
`access_requests`	Запросы квоты или роли и решение администратора
`quotas`	Занятый, зарезервированный и общий объём, тариф
`audit_records`	Журнал действий только на добавление
`notifications`	Уведомления пользователя и признак прочтения
`comments`	Комментарии к файлу с автором и временем
`upload_sessions`	Сессия загрузки частями, ключ идемпотентности и статус

3.1 Ключевые правила

- Публичная ссылка имеет ровно одного адресата: файл или папку. Это правило проверяется ограничением `num_nonnulls(file_id, folder_id) = 1`.
- Публичная ссылка и членство в общей папке — разные механизмы. Ссылка даёт гостю просмотр или скачивание, а редактирование возможно только для участника папки с соответству-

ющей ролью.

- Восстановление версии создаёт новую запись `file_versions`; старая версия не перезаписывается. Объекты S3 адресуются уникальным `storage_key`.
- Перенос файла в корзину отзывает ссылки; возврат из корзины не активирует их повторно. Карантин запрещает публикацию.
- Квота учитывает занятый и зарезервированный объём; уменьшение ниже занятого объёма запрещено. Решение по запросу применяется однократно.
- Журнал аудита доступен только для добавления; изменение и удаление записей запрещены триггером.
- Иерархия папок не содержит циклов; одинаковые имена в одной папке запрещены без учёта регистра.

Полный перечень атрибутов приведён в `database/logical-model.md`, исходная диаграмма — в `diagrams/logical.puml`.

4 Физическая модель данных

Физическая модель реализована для PostgreSQL 16. Идентификаторы имеют тип `uuid`, время — `timestamptz`, размеры — `bigint`. Для ссылок, сессий и приглашений хранятся хеши токенов; пароли хранятся отдельно в виде хешей. Полная DDL находится в `database/schema.sql`, проверки ограничений — в `database/test_constraints.sql`.

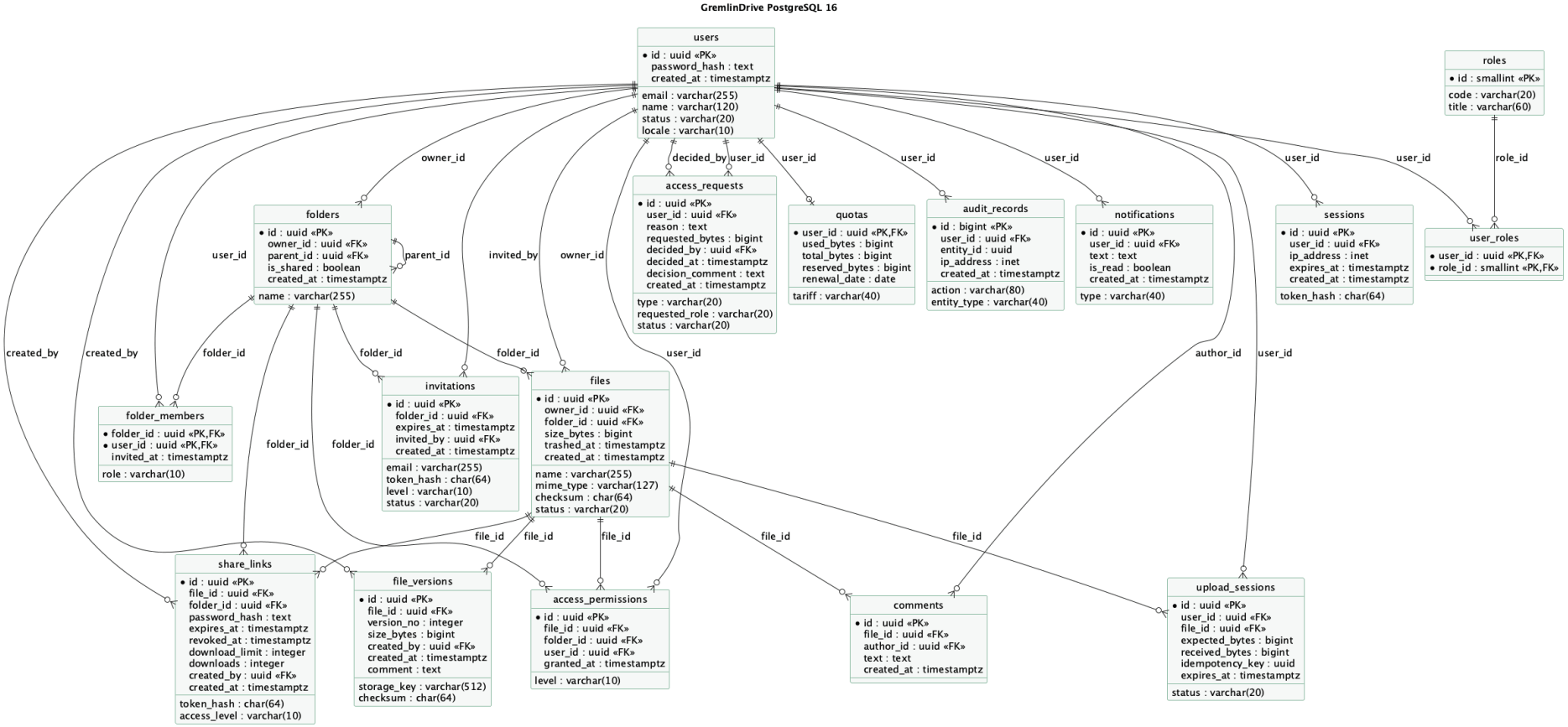


Рис. 3 – Физическая модель PostgreSQL: 17 таблиц, ключи и связи

4.1 Ограничения и индексы

Таблица 5 – Ключевые ограничения физической модели

Ограничение	Смысл
`quotas_check`	`used_bytes + reserved_bytes <= total_bytes`
`share_links_target`	Ровно один адресат: `file_id` или `folder_id`
`folders_sibling_name`	Уникальное имя в папке без учёта регистра, включая корень
`files_folder_owner`	Составной внешний ключ не позволяет положить файл в чужую папку
`files_trashed_at`	Признак `trashed` и дата удаления согласованы
`access_requests_decision`	Решение требует администратора и времени; отказ требует комментария
`folder_no_cycles`	Триггер запрещает циклы в иерархии папок
`audit_no_change`	Журнал аудита допускает только добавление записей
`file_versions_unique`	Номер версии уникален в пределах файла
`sessions_expiry`	Срок сессии больше времени создания

Индексы ускоряют выборку файлов по владельцу и папке, поиск по имени без учёта регистра, очередь запросов квоты, непрочитанные уведомления, комментарии и аудит по времени. Для ссылок и приглашений используются уникальные индексы по хешам токенов.

4.2 Фрагменты DDL

Ниже приведены ключевые таблицы; полная схема включает также роли, сессии, приглашения, квоты, аудит, уведомления, комментарии и сессии загрузки.

```
CREATE TABLE users (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 email varchar(255) NOT NULL UNIQUE,
 password_hash text NOT NULL,
 name varchar(120) NOT NULL,
 status varchar(20) NOT NULL DEFAULT 'pending'
   CHECK (status IN ('pending','active','blocked')),
 created_at timestamptz NOT NULL DEFAULT now(),
 CHECK (email = lower(email))
);

CREATE TABLE files (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 owner_id uuid NOT NULL REFERENCES users(id),
 folder_id uuid,
 name varchar(255) NOT NULL
   CHECK (length(trim(name)) > 0 AND name !~ '[/\\]'),
 size_bytes bigint NOT NULL CHECK (size_bytes >= 0),
 checksum char(64),
 status varchar(20) NOT NULL DEFAULT 'pending'
   CHECK (status IN ('pending','active','quarantine','trashed')),
 trashed_at timestamptz,
 created_at timestamptz NOT NULL DEFAULT now(),
 UNIQUE (id, owner_id),
 FOREIGN KEY (folder_id, owner_id) REFERENCES folders(id, owner_id),
 CHECK ((status = 'trashed') = (trashed_at IS NOT NULL))
);

CREATE TABLE share_links (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 file_id uuid REFERENCES files(id) ON DELETE CASCADE,
 folder_id uuid REFERENCES folders(id) ON DELETE CASCADE,
 token_hash char(64) NOT NULL UNIQUE,
 access_level varchar(10) NOT NULL
   CHECK (access_level IN ('view','download')),
 password_hash text,
```

```
 expires_at timestamptz,
 revoked_at timestamptz,
 download_limit integer CHECK (download_limit > 0),
 downloads integer NOT NULL DEFAULT 0 CHECK (downloads >= 0),
 created_by uuid NOT NULL REFERENCES users(id),
 created_at timestamptz NOT NULL DEFAULT now(),
 CHECK (num_nonnulls(file_id, folder_id) = 1),
 CHECK (expires_at IS NULL OR expires_at > created_at),
 CHECK (download_limit IS NULL OR downloads <= download_limit)
);
```

Listing 1 — Фрагмент физической схемы: пользователи, файлы и ссылки

Ограничения базы не заменяют авторизацию: доступ пользователя, резервирование квоты и переходы состояний дополнительно проверяются в транзакциях приложения. Раздельные физические диаграммы контекстов «пользователи и управление», «файлы и загрузка», «общий доступ» находятся в `diagrams/physical-*.png`.

5 Маршруты и контракт API

Базовый путь — `/api/v1`. Спецификация OpenAPI 3.1 хранится в `openapi.json`. HTTP-методы отражают работу с ресурсом: GET читает, POST создаёт ресурс или объект операции, PUT заменяет настройки, PATCH меняет отдельные поля, DELETE отзывает доступ или перемещает файл в корзину. Коллекции файлов поддерживают `q`, `status`, `limit` от 1 до 100 и `offset`.

Контракт содержит 52 операции. Prism на порту 4010 симулирует все операции; FastAPI на порту 8044 реализует 21 операцию с состоянием SQLite. Поле `x-implementation` показывает границу: `stateful-mock` или `contract-mock`. Промышленная регистрация, TOTP, передача частей файла, LDAP и push описаны контрактом и не выдаются за реализованные в mock.

Таблица 6 – Связь вариантов использования с основными операциями API

UC	Основные операции	Реализация
UC-01	`POST /uploads`, `PUT /uploads/{id}/parts/{no}`, `POST /uploads/{id}/completions`	contract
UC-02	`GET/POST/DELETE /share-links`	stateful
UC-03	`POST /folders/{id}/invitations`, `POST /invitations/{id}/acceptances`	contract
UC-04	`GET /files/{id}/versions`, `POST /files/{id}/version-restorations`	mixed
UC-05	`GET /me/quota`, `POST /access-requests`, `GET/POST /admin/access-requests`, `PUT /admin/users/{id}/quota`	stateful
UC-06	`POST /auth/registrations`, `POST /auth/email-confirmations`	contract
UC-07	`GET /me`, `POST /auth/sessions`, `DELETE /auth/sessions/current`, `POST /auth/password-resets`	mixed

UC	Основные операции	Реализация
UC-08	`POST /auth/totp/enrollments`, `POST /auth/totp/verifications`	contract
UC-09	`GET/POST /folders`, `PATCH /files/{id}`	stateful
UC-10	`GET /files`, `GET /files/{id}`, `GET /files/{id}/content`, `GET /files/{id}/previews`	mixed
UC-11	`DELETE /files/{id}`, `POST /files/{id}/restorations`	stateful
UC-12	`POST /public/shares/{token}/access`, `GET /public/shares/{token}/content`	stateful
UC-13	`GET/PUT/DELETE /folders/{id}/members/{user}`	contract
UC-14	`GET/POST /files/{id}/comments`, `GET /notifications`, `PATCH /notifications/{id}`	contract
UC-15	`GET/PATCH /admin/users`, `PUT /admin/users/{id}/roles`	mixed
UC-16	`GET /admin/audit-records`, `POST /admin/reports`	mixed
UC-17	`POST /devices`, `GET /sync/changes`	contract
UC-18	`POST /admin/ldap-synchronizations`	contract
UC-19	`GET /health`	stateful

5.1 Пример запроса и ответа

```http
POST /api/v1/share-links HTTP/1.1
Authorization: Bearer lab-owner-token
Content-Type: application/json

{
  "file_id": "20000000-0000-4000-8000-000000000001",
  "password": "laboratory123",
  "expires_in_days": 7,
  "access_level": "download"
}
```

Listing 2 — Создание публичной ссылки

Успешный ответ имеет статус 201 и содержит `id`, `file_id`, `token`, `url`, `expires_at` и `access_level`. Пароль в ответ не включается. Для всех POST, PUT и PATCH с телом заданы схема и пример; для списков и деталей заданы схемы ответов.

5.2 Коды ответов

Таблица 7 – Основные коды ответов API

Код	Значение
200	Чтение или изменение выполнено
201	Создан ресурс или результат операции
401	Нет действующей сессии либо неверен пароль ссылки
403	Пользователь определён, но действие запрещено
404	Объект не найден или публичная ссылка недействительна
409	Конфликт состояния, повторное решение или неподходящая квота
413	Часть файла превышает допустимый размер
422	Запрос не соответствует схеме
429	Превышен лимит попыток проверки пароля
507	Не хватает квоты для загрузки

6 Проверка API и схемы данных

Сначала OpenAPI проверен валидатором спецификации. Затем все 52 операции вызваны на Prism; успешные JSON-ответы проверены по схемам. Некорректный запрос к `share-links` возвращает 422, пример запрета доступа — 403. Эти проверки подтверждают согласованность контракта и примеров, а бизнес-правила отдельно проверяются на stateful mock.

Таблица 8 – Результаты проверки ЛР4

Проверка	Результат
OpenAPI 3.1	52 операции, документ валиден
Prism	52 успешных ответа, схемы ответов совпадают
Bruno	52 запроса и 52 успешных теста в окружении `contract`
Pytest	22 теста, покрытие 87,5% (315 из 360 строк)
PostgreSQL	17 таблиц; 8 проверок ограничений пройдены
MCP	7 tools, 3 resources, 1 resource template, 2 prompts; stdio-тест пройден
Трассировка	52 операции связаны с UC, FR и US; `check_continuity.py` — OK

Pytest проверяет гостевой доступ, пароль и ограничение попыток, истечение и отзыв ссылки, запрет записи для читателя, отсутствие доступа у постороннего, корзину, карантин, однократное решение по квоте и отсутствие секретов в аудите. Проверки PostgreSQL подтверждают ограничения квоты, единственного адресата ссылки, имён папок, внешнего ключа, даты удаления, комментария отказа, циклов папок и неизменяемости аудита. Тестовая транзакция отменяется после проверок.

6.1 Воспроизведение

Команды выполняются в backend-репозитории `hci-10-backend-api-group-3`:

```
cd runtime
python3.12 -m venv .venv
.venv/bin/pip install -r requirements.txt
.venv/bin/pytest -q --cov=store --cov=api
```

```
.venv/bin/python test_mcp.py
```

Listing 3 — Локальные проверки

Для проверки контракта из корня репозитория запускается `docker compose up -d --build api database contract`. Затем из `runtime` выполняются `build_contract.py` и `check_contract.py`, а из `bruno` — `npx --yes @usebruno/cli run --env contract`. Проверки PostgreSQL выполняются через `psql` в контейнере `database` с файлом `database/test_constraints.sql`.

Протоколы `pytest.txt`, `contract-tests.json`, `postgresql.txt`, `bruno-summary.json` и `mcp-protocol.json` приложены к работе. Результаты относятся к учебному стенду; производительность промышленного хранилища не измерялась.

7 Проектирование и реализация MCP

MCP-сервер позволяет ассистенту находить доступные файлы, читать метаданные, создавать папки и защищённые ссылки, отзывать ссылки и работать с корзиной. Используется официальный MCP Python SDK 1.30.0 и транспорт stdio. Учебная база SQLite общая с REST mock при одинаковом `GREMLIN_DB`.

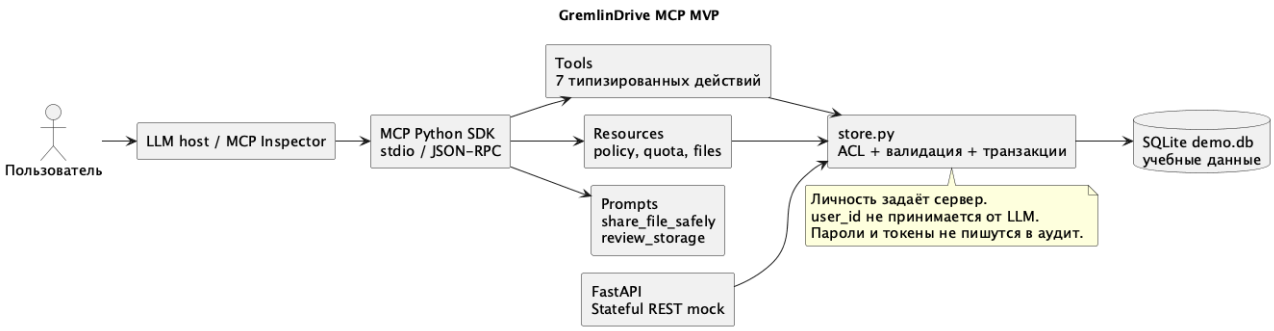


Рис. 4 – Архитектура MCP-сервера GremlinDrive

Личность пользователя задаёт переменная окружения `GREMLIN_USER_ID`; инструменты не принимают `user_id` от LLM. Каждое действие проходит проверки доступа в `store.py`. Пользователь с ролью просмотра не может создать ссылку или удалить файл. Если переменная не задана, сервер завершает работу с ошибкой, а не использует владельца по умолчанию.

7.1 Инструменты

Таблица 9 – Инструменты MCP-сервера

Инструмент	Параметры и ограничения
`list_files`	`query`, `limit` 1–100; доступные активные файлы
`get_file`	`file_id` UUID; метаданные с проверкой ACL
`create_folder`	`name`, `parent_id` или `null`; нужна запись в родительскую папку
`create_share_link`	`file_id`, пароль от 8 символов, срок 1–30 дней или `null` (14 по умолчанию), уровень `view/download`
`revoke_share_link`	`share_id` UUID; отзыв при праве записи
`trash_file`	`file_id`, `confirm=true`; корзина и отзыв ссылок
`restore_file`	`file_id`; возврат из корзины без публикации старых ссылок

Для инструментов опубликованы входные JSON-схемы (`inputSchema`); структура успешных ответов описана в коде и проверяется интеграционным тестом. Некорректный UUID и `confirm=false` отклоняются до выполнения действия.

7.2 Ресурсы и промпты

Статические ресурсы `gremlin://policy`, `gremlin://quota` и `gremlin://files/recent` дают правила, текущую квоту и список доступных файлов. Шаблон `gremlin://files/{file_id}` предоставляет метаданные конкретного файла с проверкой доступа. В MVP ресурс `recent` возвращает первые 20 файлов, отсортированных по имени.

Промпт `share_file_safely(filename)` задаёт последовательность: прочитать правила, найти файл, уточнить выбор, получить пароль и срок, создать ссылку. Промпт `review_storage` читает квоту и доступные файлы и предлагает варианты очистки. Промпт не выдаёт полномочий: проверки в сервисе выполняются независимо от текста запроса.

7.3 Проверка

`test_mcp.py` подключается как настоящий MCP-клиент по stdio, выполняет `initialize`, discovery, читает ресурсы и шаблон, получает оба промпта и вызывает все инструменты. Дополнительно проверяются отказ публикации для читателя, неверный UUID и отсутствие подтверждения. Официальный Inspector отдельно выполняет `tools/list`. Протоколы находятся в `evidence/mcp-protocol.json` и `evidence/inspector-tools.json`.

8 Вклад участников

Работа выполнена командой GremlinDrive. Ниже зафиксировано распределение задач и результаты, на которые опирается отчёт.

Таблица 10 – Распределение работ по ЛР4

Участник	Зона ответственности	Результат и коммиты
Матвей Вержбицкий	Требования и прецеденты, сохранение UC-01–UC-05, логическая модель данных, проверка преемственности, ревью API и MCP	`392a849`: SRS, User Stories, UML и архитектура ЛР3; логическая модель и реестр UC ЛР4
Александр Сытин	Физическая модель PostgreSQL, OpenAPI, Bruno/Postman, stateful mock, MCP-сервер, тесты, CI и оформление отчёта	`4f1cd6b`, `073634e`: DDL, контракт, mock, MCP, проверки и документация
Совместно	Согласование контракта, проверка результатов, приёмка комплекта	Переработка ЛР4: `16c62c9` (backend), `6c5b292` (docs)

Историю и полные сообщения коммитов можно посмотреть в репозиториях:

- https://github.com/fpmi-hci-2026/hci-10-project-group-4/commits/main
- https://github.com/fpmi-hci-2026/hci-10-backend-api-group-3/commits/main

9 Выводы

В ходе лабораторной работы №4 для GremlinDrive спроектирован REST API на 52 операции, построены логическая и физическая модели PostgreSQL на 17 таблиц, реализован stateful mock на Python/SQLite для 21 операции и MCP-сервер с семью инструментами, тремя ресурсами, одним шаблоном ресурса и двумя промптами.

Сохранена преемственность ЛР3: роли, User Stories, UC-01–UC-05 и сущности данных не переопределены. Контракт связан с требованиями через `x-use-case`, `x-requirements` и `x-user-stories`; трассировка проверяется отдельным скриптом. Ссылка отделена от членства в общей папке, восстановление версии создаёт новую версию, а восстановление из корзины не активирует старые ссылки.

Проверки подтверждают работоспособность учебного стенда: 22 теста с покрытием 87,5%, 52 операции Prism, 52 запроса Bruno, восемь ограничений PostgreSQL и реальный обмен MCP по stdio. При этом Prism и SQLite являются учебными mock-сервисами, а не промышленным backend. Целевой стек NestJS/PostgreSQL/MinIO, реальная передача файлов, SMTP, LDAP и push остаются следующими этапами.

Следующий шаг — подключение веб- и мобильного клиентов к контракту, перенос stateful-сценариев на целевой backend и развитие MCP-интерфейса без дублирования доменных проверок.

10 Список литературы

1. Давидовская М.И. Проектирование человеко-машинных интерфейсов: методические указания к лабораторной работе №4 «Стили интеграции программных систем и проектирование API». Минск: БГУ, 2026.
2. OpenAPI Initiative. OpenAPI Specification 3.1.0. https://spec.openapis.org/oas/v3.1.0.
3. Model Context Protocol. Specification and Python SDK. https://modelcontextprotocol.io.
4. FastAPI. Documentation. https://fastapi.tiangolo.com.
5. PostgreSQL Global Development Group. PostgreSQL 16 Documentation. https://www.postgresql.org/docs/16/.
6. Docker Inc. Docker Compose Documentation. https://docs.docker.com/compose/.
7. Bruno. API Client and CLI Documentation. https://docs.usebruno.com.
8. PlantUML. Language Reference Guide. https://plantuml.com/guide.